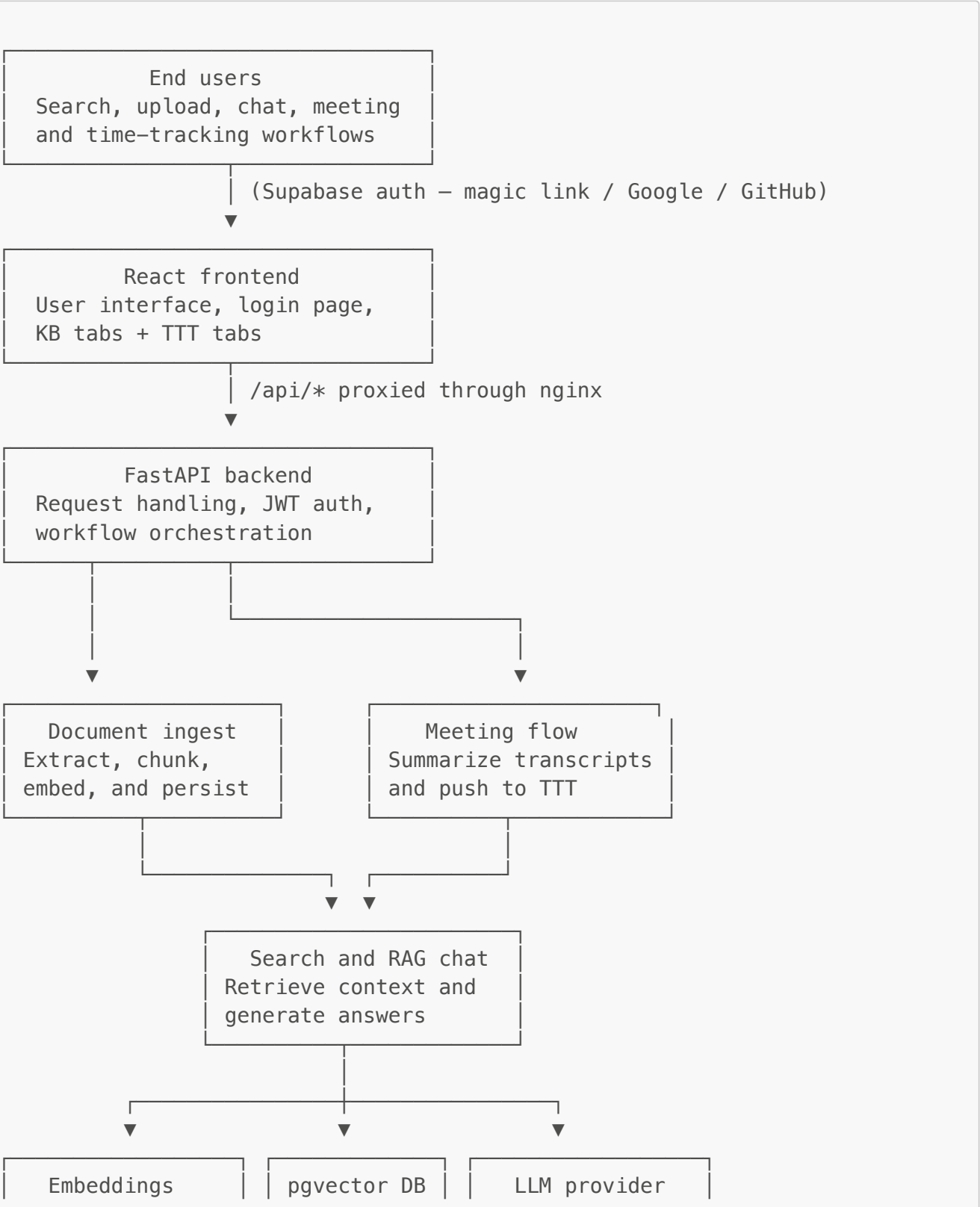
WorkTrace — General Overview

WorkTrace is a document and meeting intelligence platform. Upload any file, ask questions in natural language, and automatically log meeting summaries to a built-in time tracker — all behind per-user authentication, running on OpenShift with in-cluster PostgreSQL + pgvector.

Platform Workflow



```
  │ Nomic / Ollama   │ │ Chunks and  │ │ watsonx / Groq / │
  │                  │ │ metadata    │ │ OpenAI / Ollama  │
  └──────────────────┘ └─────────────┘ └──────────────────┘
```

Features

Feature	Description
Multi-format ingestion	PDF, DOCX, TXT, MD, CSV, JSON, YAML, HTML, images (OCR), source code
Semantic search	pgvector cosine similarity — find content by meaning, not keywords
RAG chat	Multi-turn conversational Q&A grounded in your indexed documents
Agentic meeting summariser	5-step agent loop (search KB → look up TTT history → classify → synthesise → push); full tool-call trace displayed in the UI
LangChain pipeline	Optional drop-in for RAG chat and agentic summarisation — activate with `USE_LANGCHAIN=true`; custom implementation always kept as fallback
Chat feedback loop	Thumbs up/down on every assistant message; approval score + low-rated query log in the Feedback tab
Meeting intelligence	Ingest transcripts, auto-extract metadata, generate structured summaries
Time Task Tracker (TTT)	Meeting summaries auto-pushed to `time_entries`; dashboard, manual entry, reports, CSV export, calendar import
Multi-user isolation	Every document and time entry is scoped to the authenticated Supabase user
Pluggable LLM	watsonx · OpenAI · Groq · Ollama — switch via one environment variable
OpenShift deployment	One script deploys everything — postgres, backend, frontend, secrets
Cluster migration	`dump.sh` + `deploy.sh` auto-restore from backup when moving to a new cluster

---

Key Workflows

1. Document ingestion

Files are submitted through the Upload tab or the `/upload` API endpoint. The backend automatically detects the file type, extracts text, splits it into overlapping chunks, generates vector embeddings, and stores everything in pgvector. Once indexed, the file's content is immediately available in search and chat.

Supported file types:

Category	Formats
Documents	PDF · DOCX · DOC · TXT · MD · RST
Data	CSV · JSON · YAML · XML · HTML
Code	PY · JS · TS · GO · JAVA · C · C++ · RB · SH
Images	PNG · JPG · JPEG · GIF · BMP · TIFF · WEBP (OCR via Tesseract)
Meetings	TXT · MD · VTT · SRT · PDF · DOCX with structured header

Already-indexed files are skipped automatically. A **Force re-index** option re-processes a file from scratch.

2. Semantic search

The Search tab accepts natural-language queries. The query is embedded and compared against stored chunk vectors using cosine similarity. Results include source path, relevance score, and a highlighted snippet. Filters let users narrow by file type, source filename, or result count. Snippets can be expanded to show the full chunk.

3. Retrieval-augmented chat

The Chat tab is a multi-turn conversational interface grounded in indexed content. Each question triggers a vector search to retrieve relevant chunks, which are assembled into a grounded prompt alongside conversation history. The configured LLM generates an answer and returns it with cited source references. The system is constrained to answer only from retrieved context — it does not fall back on the model's base knowledge.

When a question references time entries ("hours logged", "what did I work on", "billable this week"), the RAG pipeline additionally fetches structured rows from the Time Task Tracker database and injects them as context.

4. Meeting summarization

The Meeting Upload tab accepts meeting transcripts. Supported formats include TXT, MD, VTT, SRT, PDF, and DOCX. Two summarisation modes are available:

Standard RAG mode

1. **Ingest** — the transcript is indexed like any other document (fast, seconds)
2. **Summarize** — a single RAG call generates a summary covering topics, decisions, and action items

Agentic mode

Runs a multi-step agent pipeline that produces a richer summary by combining document retrieval with historical project context. Two implementations are available, selected by the `USE_LANGCHAIN` flag:

Custom pipeline (`USE_LANGCHAIN=false`, default) — fixed 5-step sequence:

Step	Tool	What it does
1	`search_kb`	Retrieves the most relevant transcript chunks from pgvector
2	`lookup_ttt`	Fetches past TTT entries for the same project for historical context
3	`classify`	Infers project code, task type, and billable flag from the filename and organizer
4	`synthesise`	Calls the LLM with all gathered context (chunks + history)
5	`push_ttt`	Inserts the completed time entry into the Time Task Tracker

LangChain pipeline (`USE_LANGCHAIN=true`) — `AgentExecutor` with the same three tools (`search_kb`, `lookup_ttt`, `push_to_ttt`). The LLM dynamically decides which tools to call, in what order, and whether to retry. The `classify` step is removed — the LLM infers project code from context.

After completion, the full agent trace (each tool's inputs and outputs) is shown in the UI so the reasoning process is transparent.

After summarization in either mode, the result is automatically pushed to the Time Task Tracker as a logged time entry.

Structured transcript header — transcripts that include a header block are auto-parsed for metadata:

```
Date: 2025-01-15
Meeting Title: Sprint Review
Organizer: jane@example.com
Attendees: Alice, Bob, Carol
Project Code: PROJ-42
Duration: 45 minutes
Billable: Yes
Platform: Teams
```

5. Time Task Tracker (TTT)

The TTT module is a full time-tracking system built into the platform. It stores entries in a `time_entries` table scoped per Supabase user.

TTT capabilities:

Capability	Description
Dashboard	Charts and stat cards showing total hours, meeting counts, and project breakdowns for any date range
Entry list	Filterable, editable list of all logged entries with bulk delete
Manual entry	Form to log any time entry (meeting or otherwise) with project code, duration, billable flag, and notes
CSV import	Upload a CSV file to bulk-import historical time entries

Capability	Description
Calendar import	Upload an ICS calendar file to import events as time entries
Reports	Date-range summary reports broken down by project and task type
CSV export	Download all entries for a date range as a CSV file
AI classification	`/ttt/classify` endpoint infers project code and billable flag from a meeting title
Chat feedback	Rate any chat response 👍/👎; the Feedback tab shows approval score, trend, and a drill-down of low-rated queries

6. Authentication and multi-user isolation

WorkTrace uses **Supabase** for authentication. All routes that read or write user data require a valid JWT in the `Authorization: Bearer` header.

Sign-in methods:

- Magic link (passwordless email)
- Google OAuth
- GitHub OAuth

The backend validates tokens using **ES256 via JWKS** (newer Supabase projects) with automatic fallback to **HS256** using the static JWT secret. Every `documents` row and every `time_entries` row is associated with the authenticated user's UUID, so users can only see and modify their own data.

Deployment

Local development

```
# 1. Start pgvector
docker compose up -d

# 2. Backend
cd backend
cp .env.example .env    # fill in values
pip install -e .
uvicorn api:app --reload --port 8000

# 3. Frontend
cd frontend
cp .env.example .env.local   # set VITE_SUPABASE_URL +
VITE_SUPABASE_ANON_KEY
npm install
npm run dev
```

- Frontend → http://localhost:5173
- API docs → http://localhost:8000/docs

OpenShift (production)

WorkTrace runs on OpenShift with an in-cluster pgvector database backed by a Ceph RBD persistent volume. A single `deploy.sh` script handles image builds, secret injection, and pod deployment.

First time:

```
cp openshift/deploy.env.example openshift/deploy.env
# Fill in: OC_SERVER, OC_TOKEN, Supabase keys, POSTGRES_PASSWORD,
watsonx/Groq keys
./openshift/deploy.sh
```

New cluster (after expiry):

```
# 1. Dump data before the cluster expires
./openshift/dump.sh

# 2. Update OC_SERVER and OC_TOKEN in deploy.env

# 3. Deploy — the script auto-restores the backup
./openshift/deploy.sh
```

After each deploy, update the **Supabase → Authentication → URL Configuration** with the new Route URL so login redirects work.

See `openshift/QUICKSTART.md` for the full step-by-step guide.

LLM providers

Switch by setting `LLM_PROVIDER` in `deploy.env` (or `.env` locally):

Provider	Key settings
watsonx	`LLM_PROVIDER=watsonx` + `WATSONX_API_KEY` + `WATSONX_PROJECT_ID` + `WATSONX_MODEL_ID`
Groq	`LLM_PROVIDER=openai` + `OPENAI_API_KEY=gsk_...` + `OPENAI_BASE_URL=https://api.groq.com/openai/v1` + `OPENAI_CHAT_MODEL=llama-3.1-8b-instant`
OpenAI	`LLM_PROVIDER=openai` + `OPENAI_API_KEY=sk-...`
Ollama (local)	`LLM_PROVIDER=ollama` + `OLLAMA_HOST` + `OLLAMA_CHAT_MODEL`

The embedding provider is configured separately via `EMBED_PROVIDER` (`nomic` or `ollama`).

LangChain pipeline (optional)

Set `USE_LANGCHAIN=true` in `deploy.env` to route the RAG chat and agentic meeting pipelines through LangChain. The custom implementation is always preserved as a fallback.

Aspect	Custom (default)	LangChain
RAG chat driver	`WatsonxProvider.chat()` / httpx	`ChatWatsonx` / `ChatOpenAI` / `ChatOllama` via LCEL pipe
Agentic tool order	Fixed 5-step sequence	Dynamic — `AgentExecutor` lets the LLM decide
Retrieval	`search()` + `query_ttt()` (unchanged)	Same

See `Technical.md` for implementation details.

System responsibilities summary

Layer	Role
Frontend	User interaction, login/logout, KB tabs (Chat, Search, Upload, Meeting, Sources, Feedback), TTT tabs — `frontend/src/App.jsx`
Backend API	Request handling, JWT auth, orchestration — `backend/api.py` + `backend/ttt_api.py`
Knowledge engine	Extraction, embedding, retrieval, chat — `backend/kb/` (+ LangChain equivalents in `lc_*.py`)
Auth layer	Supabase JWT validation (ES256 + HS256) — `backend/kb/auth.py`
Vector store	Chunks, metadata, embeddings in PostgreSQL + pgvector
TTT store	Time entries in a separate PostgreSQL database (Neon or in-cluster)
LLM layer	Configurable answer generation — `backend/kb/llm.py`

For implementation details, see `Technical.md`.